AULA 05:
ESTRUTURA DE CONDIÇÃO

Profª. Drª. Tatiele B. Moro

- Exemplo importante para entender em que casos utilizaremos `if`, `else`, `else if`, somente um deles, todos eles, alguns deles, etc.!!!

SEMPRE IRÁ DEPENDER DO PROBLEMA QUE ESTÁ SENDO RESOLVIDO!

Encadeamento de condições

- Outros nomes:
  - Desvio condicional composto;
  - Seleção encadeada; ou
  - if aninhado;

Encadeamento de condições

- É possível encadear desvios condicionais fazendo com que após um teste em que a condição testada for verdadeira, outro teste seja realizado. Este encadeamento gera estruturas como:

```
if (condição1)
{
        if (condição2)
        {
           <bloco de instruções 1>
        }
        else
        {
           <bloco de instruções 2>
        }
        <bloco de instruções 3>
}
else
{
        <bloco de instruções 4>
}
```

Exemplo 13:

```
#include <stdio.h>
main ()
{
      int num;
      printf ("Digite um numero: ");
      scanf ("%d", &num);
      if (num==10)
      {
         printf ("\n\n Voce acertou! \n");
         printf ("O numero eh igual a 10.\n");
       }
      else
      {
         if (num>10)
         {
             printf ("O numero eh maior que 10.");
         }
         else
         {
            printf ("O numero eh menor que 10.");
         }
     }
}
```

Operadores lógicos

- São operadores que geram resultados lógicos (Verdadeiro ou Falso) pela avaliação de expressões lógicas:

  - **Operador E (and):** com o uso do operador E, todos os termos da expressão devem ser verdadeiros para que a expressão seja verdadeira.

  - **Operador OU (or):** com o uso do operador OU, pelo menos um dos termos da expressão deve ser verdadeiro para que a expressão seja verdadeira.

  - **Operador NÃO (not):** inverte ou nega valor lógico de uma expressão ou elemento.

Operadores lógicos

Operador	Ação
&&	And (E)
| |	Or (OU)
!	Not (Não)

Tabela dos operadores:

NÃO (not)	
p	! p
1	0
1	0
0	1
0	1

OU (or)		
p	q	p || q
1	1	1
1	0	1
0	1	1
0	0	0

E (and)		
p	q	p && q
1	1	1
1	0	0
0	1	0
0	0	0

Exemplo 14 – número positivo e par

```
#include <stdio.h>
main ()
{
    int val;
    printf("Digite um número: ");
    scanf("%d", &val);
    if(val > 0 && val % 2 == 0)
    {
        printf("O número é par e positivo");
    }
}
```

Exemplo 15 – número positivo e par

```
#include <stdio.h>
main ()
{
    int val;
    printf("Digite um número: ");
    scanf("%d", &val);
```

`val` é maior que zero o resto da divisão de `val` por 2 é 0

```
    if(val > 0 && val % 2 == 0){
        printf("O número é par e positivo");
    }
}
```

Exemplo 16 – número positivo e ímpar

```
#include <stdio.h>
main ()
{
     int val;
     printf("Digite um número: ");
     scanf("%d", &val);
     if(val > 0 && val % 2 != 0)
     {
          printf("O número é ímpar e positivo");
     }
}
```

Exemplo 17 – número positivo e ímpar

```
#include <stdio.h>
main ()
{
     int val;
     printf("Digite um número: ");
     scanf("%d", &val);
     if(val > 0 && !(val % 2 == 0))
     {
          printf("O número é ímpar e positivo");
     }
}
```

Exemplo 18 – número positivo e ímpar

```
#include <stdio.h>
main ()
{
      int val;
      printf("Digite um número: ");
      scanf("%d", &val);
```

val é positivo ! (o resto da divisão de val por 2 é 0)

```
      if(val > 0 && !(val%2==0) ){
            printf("O número é ímpar e positivo");
      }
}
```

Exemplo 19 – número positivo e ímpar

```
#include <stdio.h>
main ()
{
      int val;
      printf("Digite um número: ");
      scanf("%d", &val);
```

val é positivo **o resto da divisão de val por 2 NÃO é 0**

```
      if(val > 0 && !(val % 2 == 0) )
      {
            printf("O número é ímpar e positivo");
      }
}
```